Drumuri minime în grafuri ponderate

Aplicaţii



- Dată o hartă rutieră, să se determine
  - un drum minim între două orașe date
  - câte un drum minim între oricare două orașe de pe hartă

Aplicaţii

- Determinarea de drumuri minime/distanţe – numeroase aplicaţii
  - probleme de rutare
  - robotică
  - procesarea imaginilor
  - strategii jocuri
  - probleme de planificare (drumuri critice)

Cadru

Fie:

- G – graf orientat ponderat
- P – drum

$$w(P) = \sum_{e \in E(P)} w(e)$$

costul/ponderea/lungimea drumului P

Cadru

Fie:

- **G – graf orientat ponderat**
- **Presupunem că G nu conține circuite de pondere negativă**

Cadru

- Fie $s, v \in V$, $s \neq v$.
- **Distanţa de la s la v**

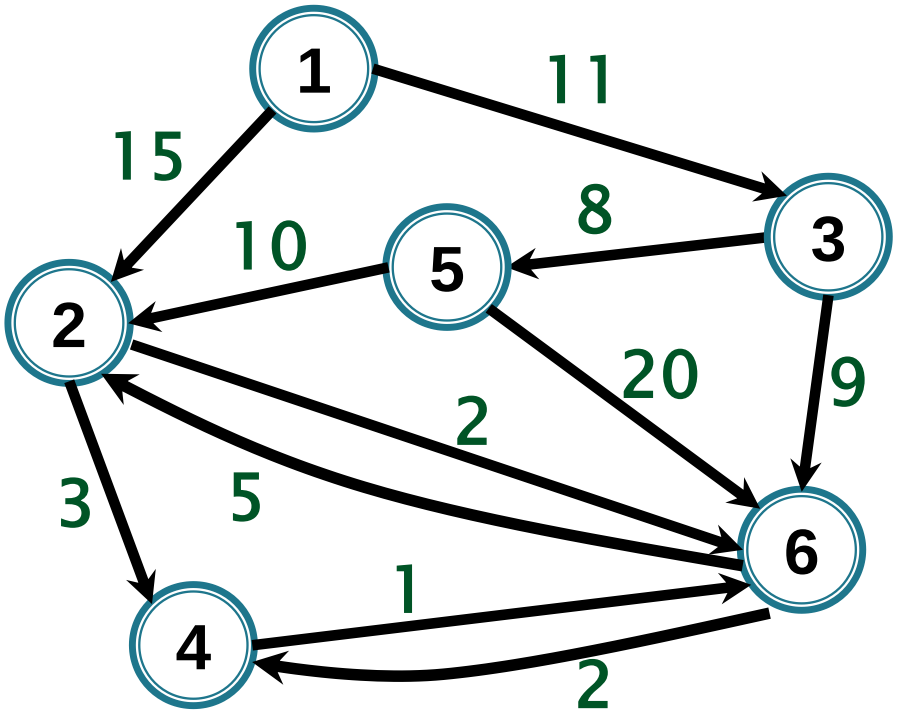
$$\delta_G(s, v) = \min\{ w(P) \mid P \text{ este drum de la } s \text{ la } v\}$$

Cadru

- Fie $s,v \in V$, $s \neq v$.
- **Distanţa de la s la v**

  $\delta_G(s, v) = \min\{ w(P) \mid P \text{ este drum de la } s \text{ la } v\}$

  - $\delta_G(s, s) = 0$
  - **Convenție.** $\min \varnothing = \infty$
- Un drum P de la s la v se numește **drum minim de la s la v** dacă $w(P) = \delta_G(s, v)$


1
11
15
8
10
5
3
2
20
9
2
3
5
6
1
4
2

Tipuri de probleme de drumuri minime

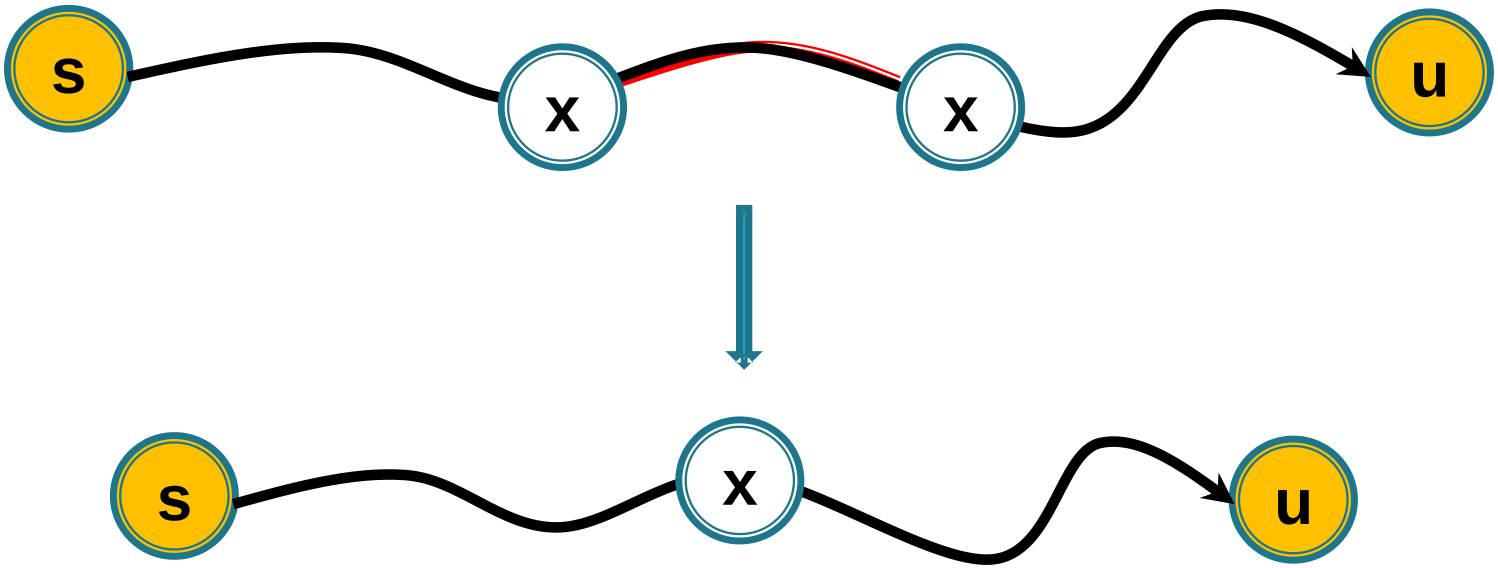
- între două vârfuri date
- de la un vârf la toate celelalte
- între oricare două vârfuri

Situaţii:

- Sunt acceptate şi arce de cost negativ?
- Graful conţine circuite?
- Graful conţine circuite de cost negativ?

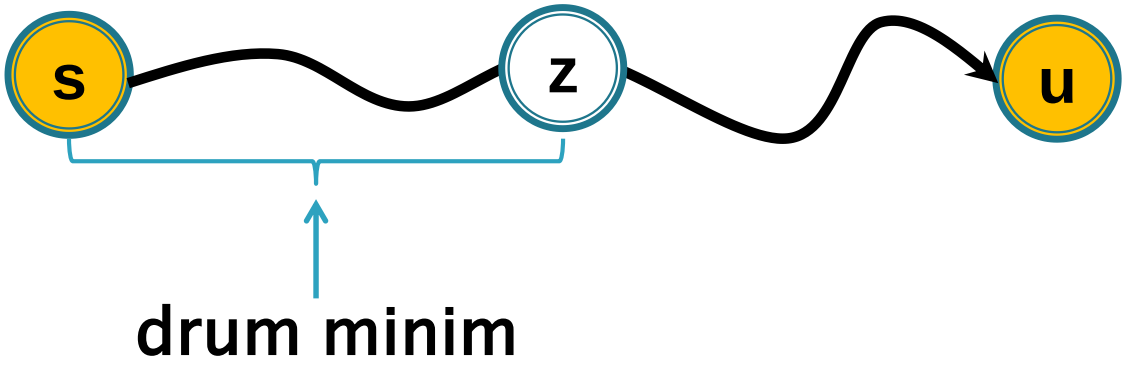
Observații

- **Observația 1.** Dacă P este un drum minim de la s la u și **nu există circuite de cost ≤ 0**, atunci P este drum **elementar** **(dacă există circuite de cost 0, P poate fi neelementar, dar există cel puțin un drum minim elementar de la s la u)**

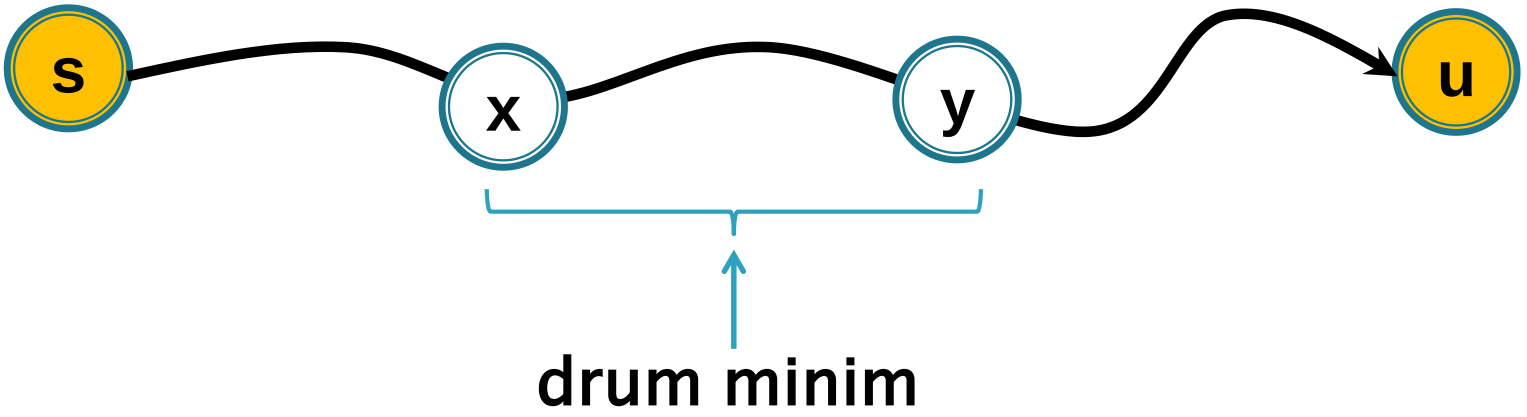


Observații

- **Observația 2.** Dacă P este un drum minim de la s la u și z este un vârf al lui P, atunci subdrumul lui P de la s la z este drum minim de la s la z.



Observații



Drumuri minime de la un vârf s dat la celelalte vârfuri (de sursă unică)

Problema drumurilor minime de sursă unică (de la s la celelalte vârfuri)

Se dau:

- un graf orientat ponderat G= (V, E, w), cu

  $w : E \to \mathbb{R}$

- un vârf de start s

Să se determine distanța de la s la fiecare vârf x al lui G / la un vârf dat t (și un arbore al distanțelor față de s/ un drum minim (elementar) de la s la t)

Drumuri minime de sursă unică s



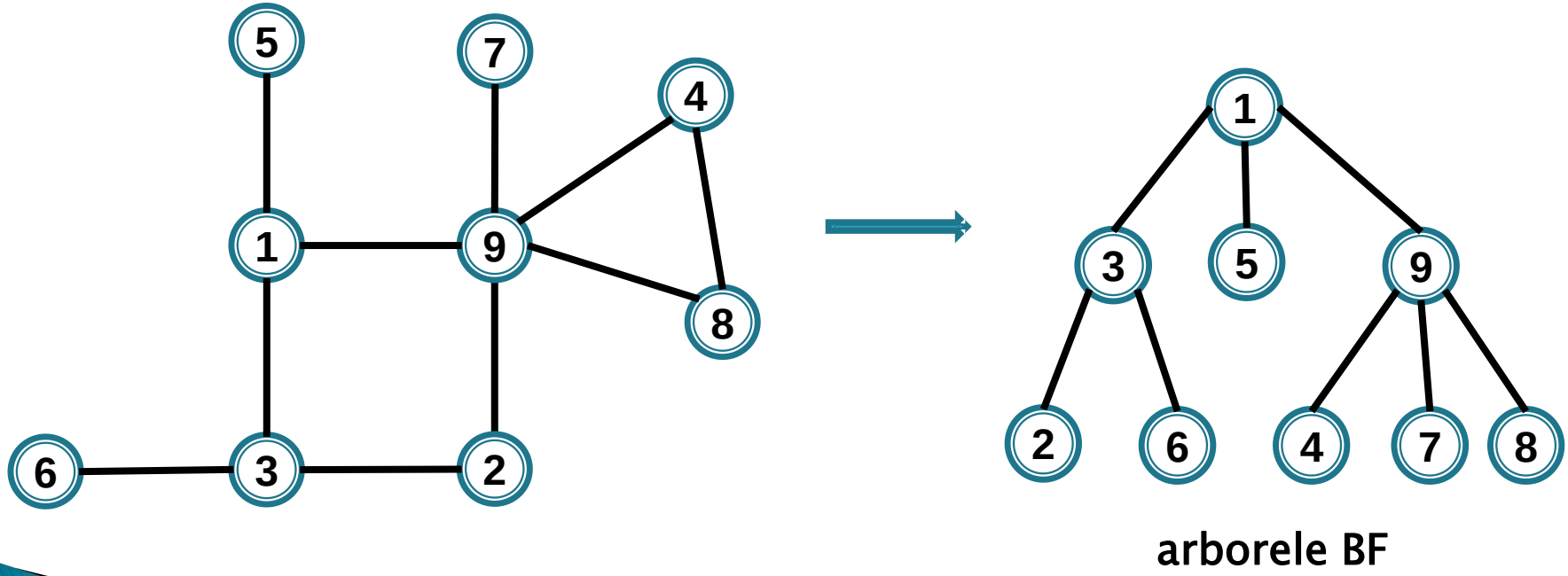
- Dacă G nu este ponderat, cum putem calcula distanţele față de s?

Drumuri minime de sursă unică s

- **Amintim**

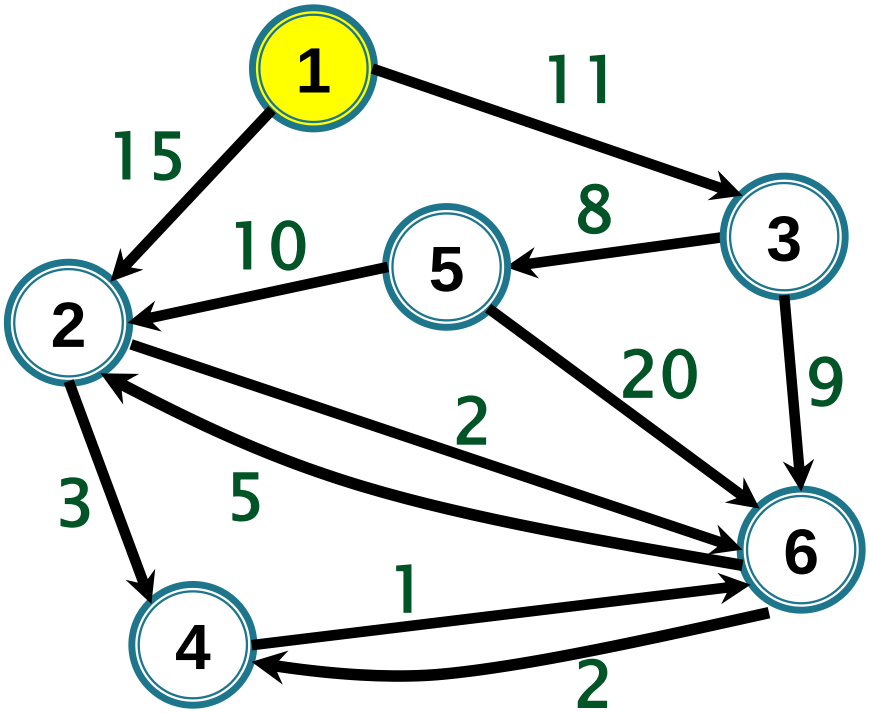
  În cazul unui graf neponderat, problema se rezolvă folosind parcurgerea BF din s

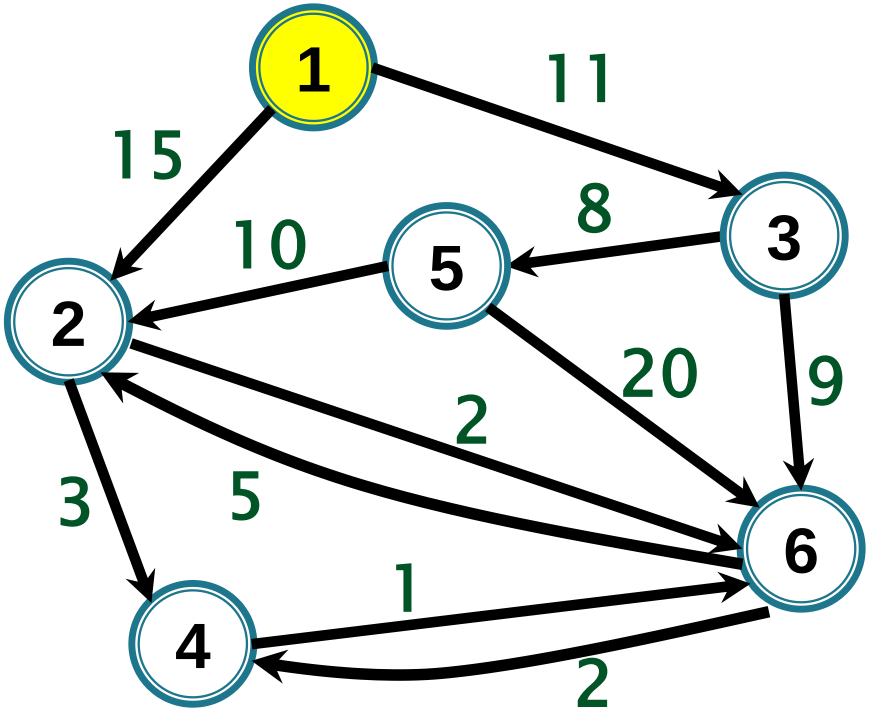
  ⇒ arborele BF (al distanțelor față de s)

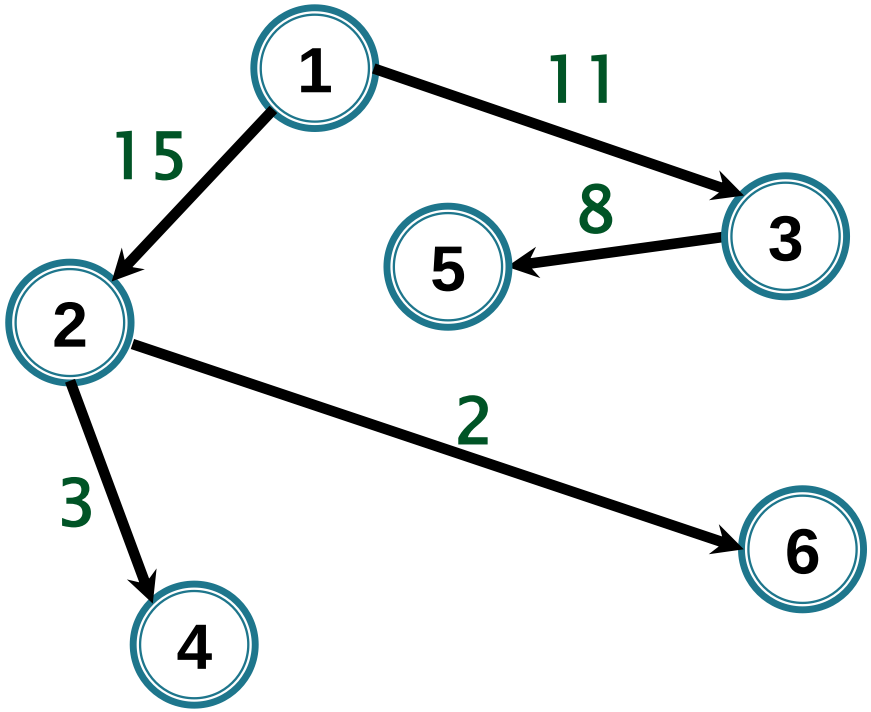


arborele BF

s = 1





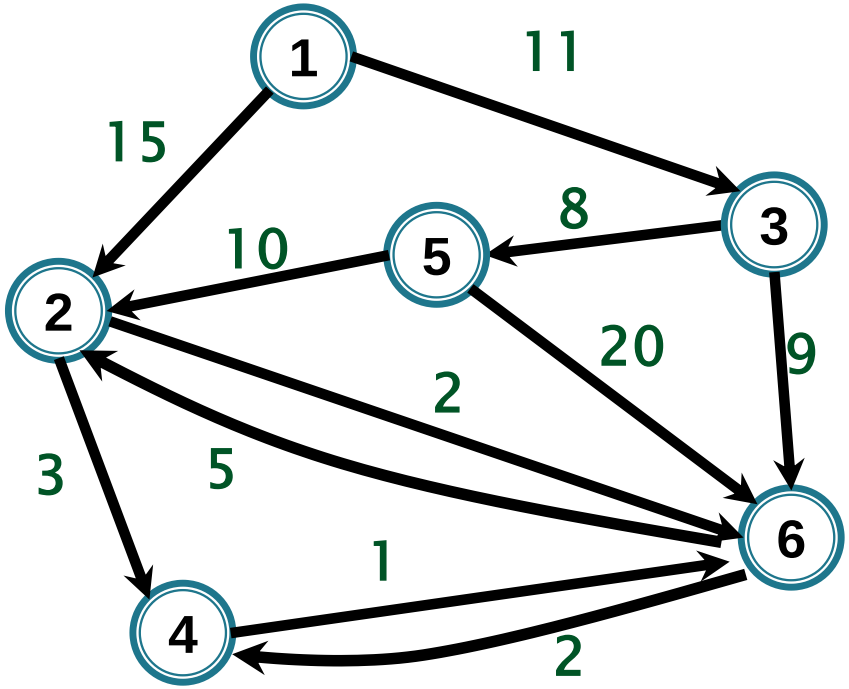


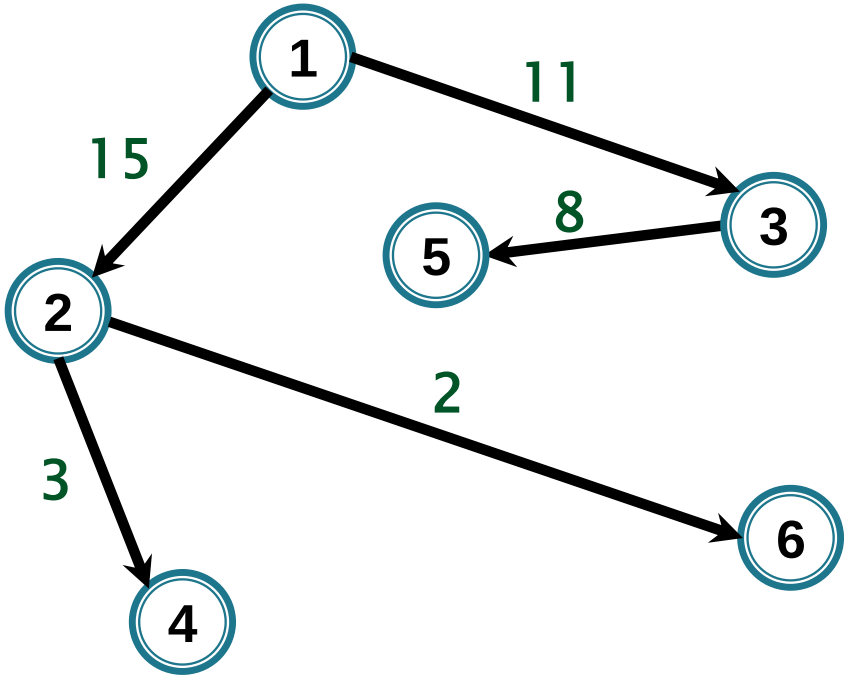
arbore al distanțelor față de 1

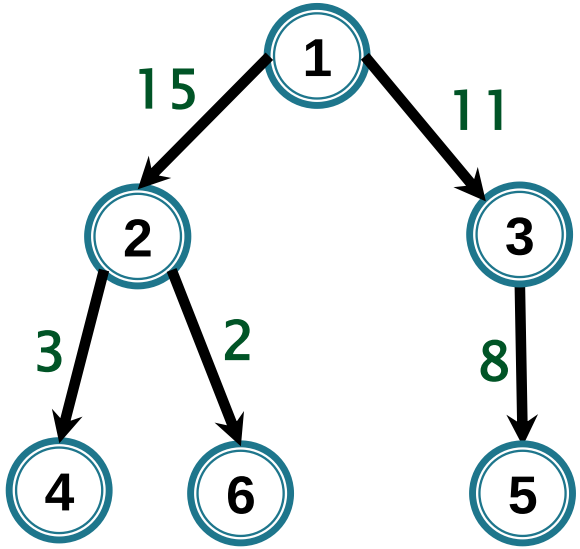
Definiție: Pentru un vârf dat s un **arbore al distanțelor față de s** = un subgraf T al lui G care **conservă distanțele** de la s la celelalte vârfuri accesibile din s

$$\delta_T(s, v) = \delta_G(s, v), \quad \forall\, v \in V \text{ accesibil din } s,$$

graful neorientat asociat lui T fiind arbore cu rădăcina în s (cu arcele corespunzătoare orientate de la s la frunze)



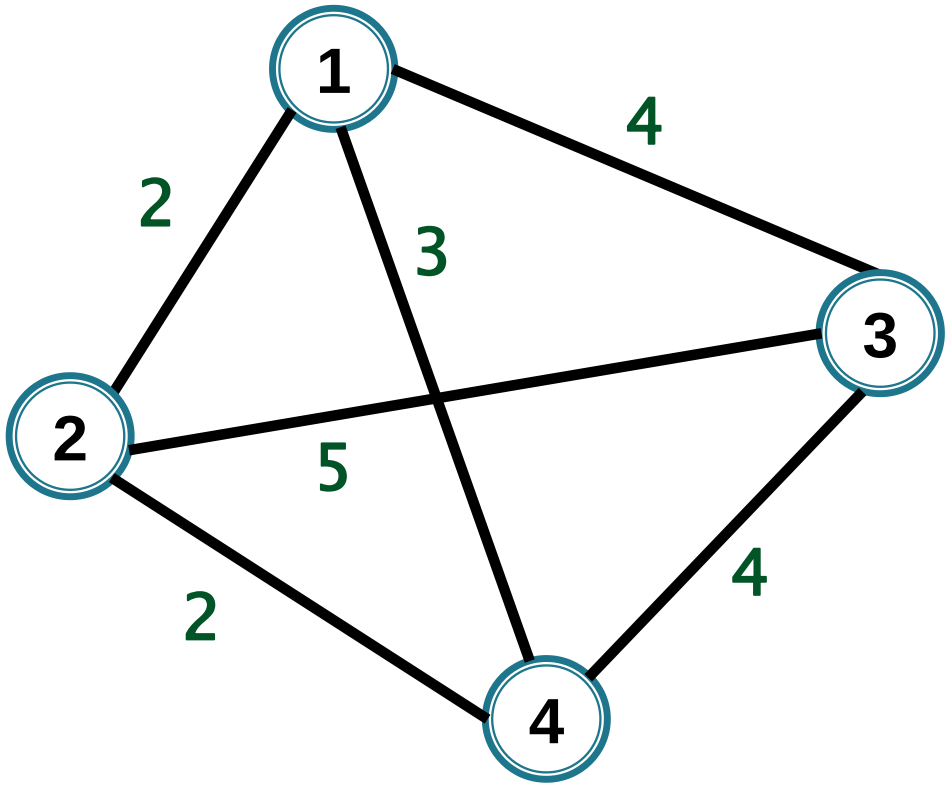




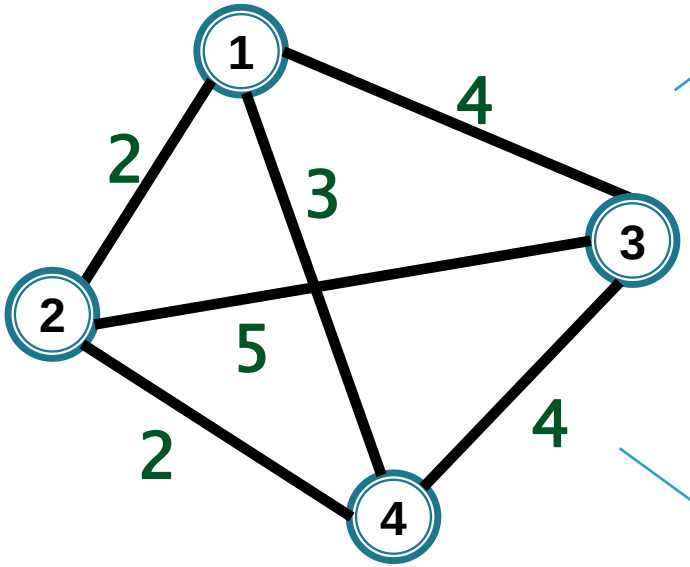
arbore al distanţelor faţă de 1

- Presupunem că toate vârfurile sunt accesibile din s
- Problema drumurilor minime de sursă unică este echivalentă cu determinarea unui arbore al distanțelor față de s

- Un arbore parțial de cost minim nu este neapărat un arbore de distanțe



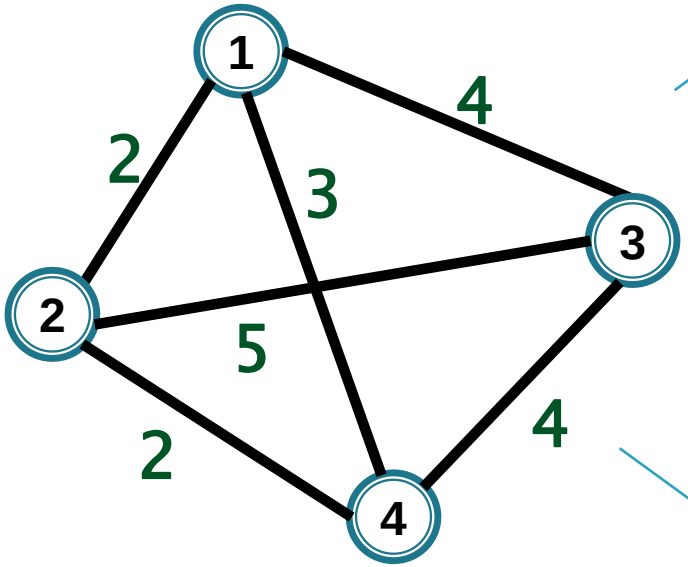
Un arbore parțial de cost minim nu este neapărat un arbore de distanțe

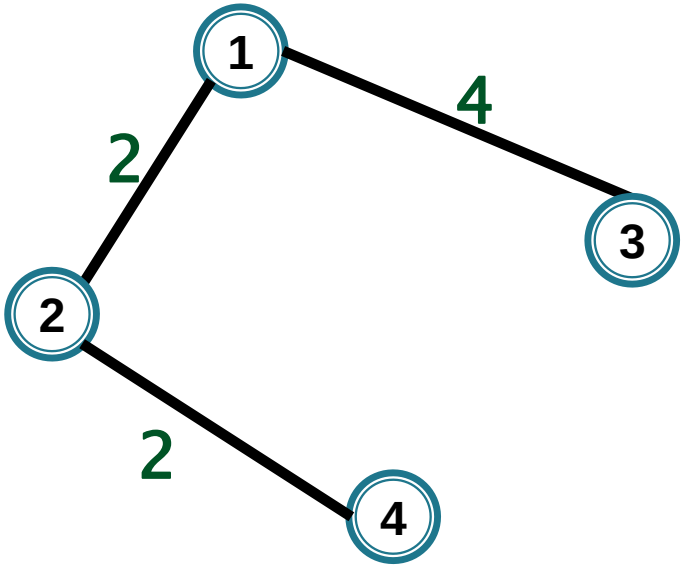


Apcm

Arbore al distanțelor față de 1

Un arbore parțial de cost minim nu este neapărat un arbore de distanțe

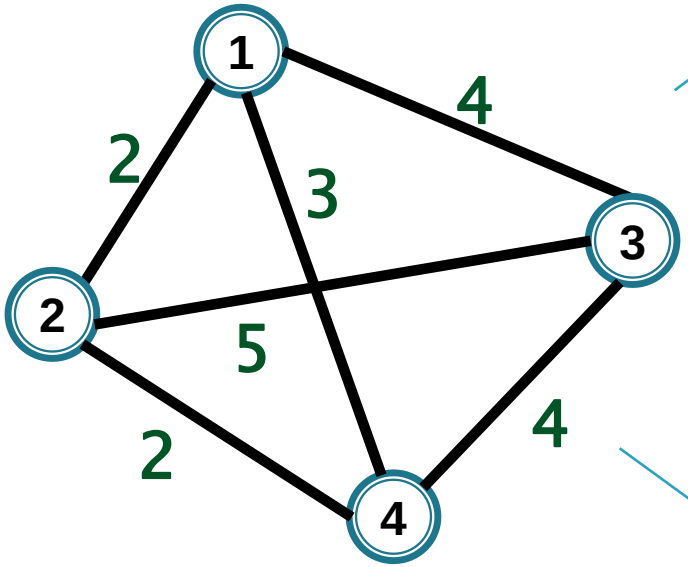


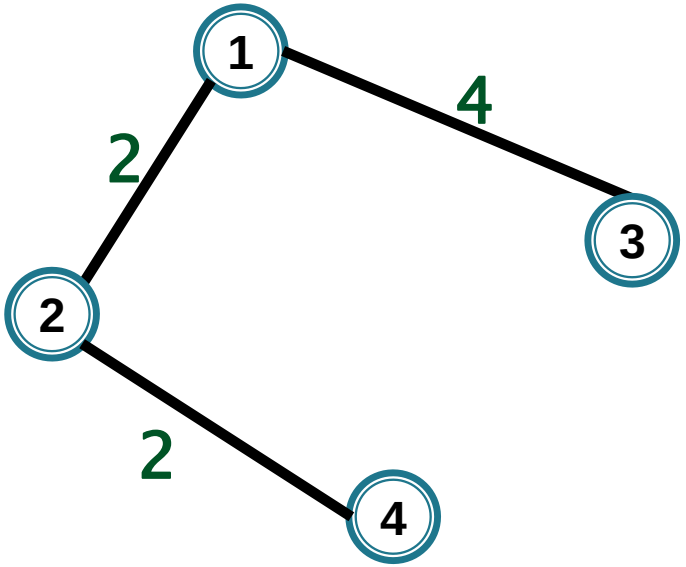


Apcm

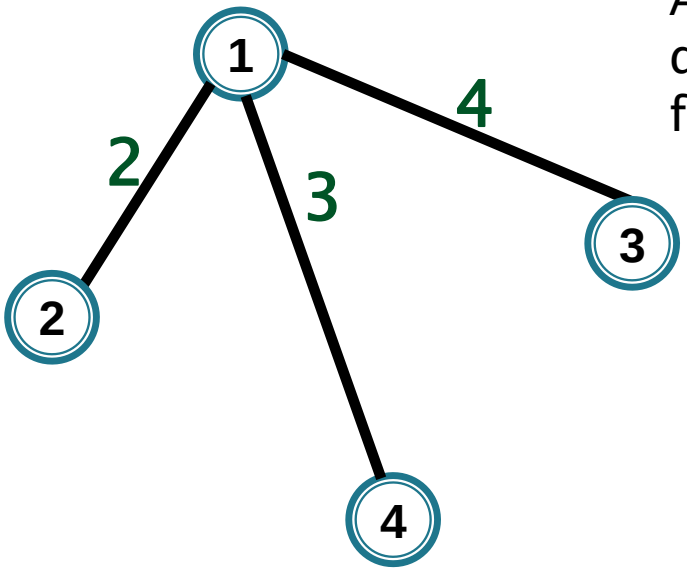
Arbore al distanțelor față de 1

Un arbore parțial de cost minim nu este neapărat un arbore de distanțe





Apcm



Arbore al distanțelor față de 1

Drumuri minime de sursă unică s



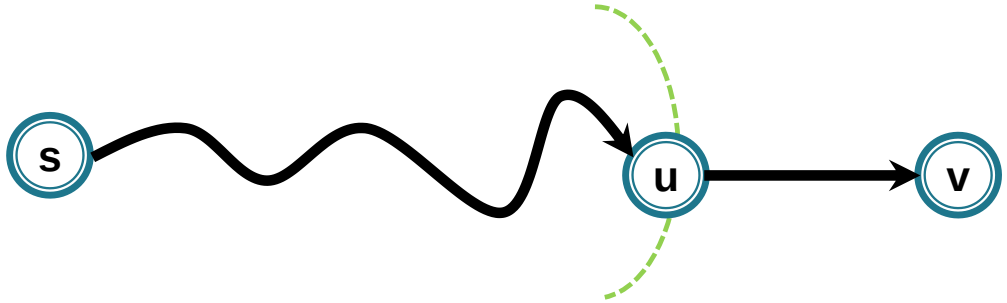
- În ce ordine considerăm vârfurile pentru a calcula distanţele față de s?

Drumuri minime de sursă unică s

- În ce ordine considerăm vârfurile pentru a calcula distanţele față de s?



"din aproape în aproape"



Dacă u este predecesor al lui v pe un drum minim de la s la v ⇒

$\delta(s,v) = \delta(s,u) + w(uv)$

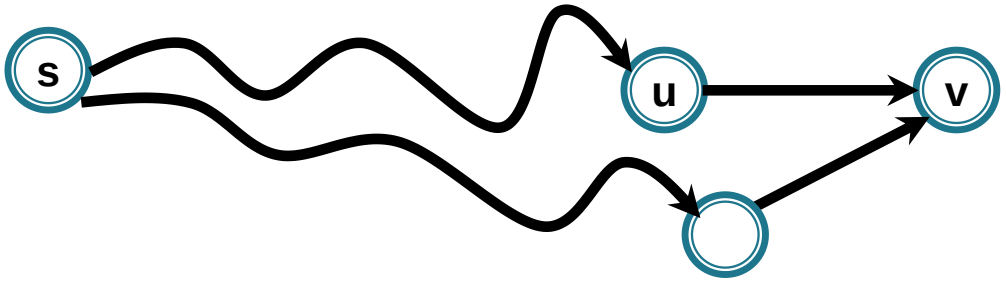
Știm $\delta(s, u)$ ⇒ aflăm și $\delta(s, v)$

Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$

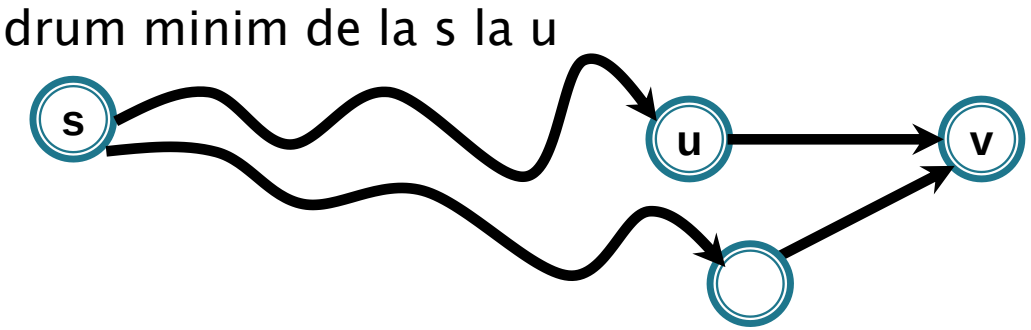
drum minim de la s la u



Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$



Deoarece:

- un drum minim de la s la predecesor u al lui v + arcul uv = un drum de la s la v (de cost $\geq \delta(s,v)$)
- Un drum minim P de la s la v = un drum minim de la s la predecesor u al lui v + arcul uv

Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$

Pentru a calcula $\delta(s,v)$ trebuie ca $\delta(s,u)$ să fie deja calculate pentru orice u predecesor al lui v =>

trebuie o ordonare a vârfurilor astfel încât dacă $uv \in E$, atunci u se află înaintea lui v

Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$

Pentru a calcula $\delta(s,v)$ trebuie ca $\delta(s,u)$ să fie deja calculate pentru orice u predecesor al lui v =>

trebuie o ordonare a vârfurilor astfel încât dacă $uv \in E$, atunci u se află înaintea lui v



Ordonarea = Sortare topologică

Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$

Pentru a calcula $\delta(s,v)$ trebuie ca $\delta(s,u)$ să fie deja calculate pentru orice u predecesor al lui v =>

trebuie o ordonare a vârfurilor astfel încât dacă $uv \in E$, atunci u se află înaintea lui v



Ordonarea = Sortare topologică



Sortare topologică – doar în grafuri fără circuite
DAG = Directed Acyclic Graphs

Drumuri minime de sursă unică s

- **Relație de recurență**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E\}$$

Există ordine de calcul a distanțelor în DAG = sortare topologică

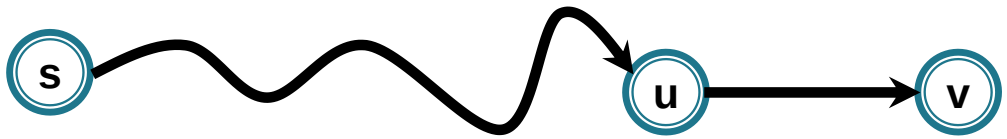


Și dacă avem circuite?

Drumuri minime de sursă unică s

- **Relație de recurență – pentru costuri pozitive**

Observație: Predecesorul u al lui v pe un s–v drum minim este mai aproape de sursa s decât v, deoarece arcele au cost pozitiv



Drumuri minime de sursă unică s

- **Relație de recurență – pentru costuri pozitive**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E,\ \delta(s,u) < \delta(s,v)\ ??\ \}$$

Predecesorul u al lui v pe un s-v drum minim este mai aproape de sursa s decât v, deoarece arcele au cost pozitiv

Drumuri minime de sursă unică s

- **Relație de recurență – pentru costuri pozitive**

$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E,\ \delta(s,u) < \delta(s,v)\ ?\ \}$$

Algoritmul lui DIJKSTRA – Vârfurile considerate în funcție de distanța **estimată** față de sursă, la fiecare pas este vizitat vârful cel mai apropiat de sursă și extinse drumurile din el

Drumuri minime de sursă unică s

- **Relație de recurență – pentru costuri pozitive**

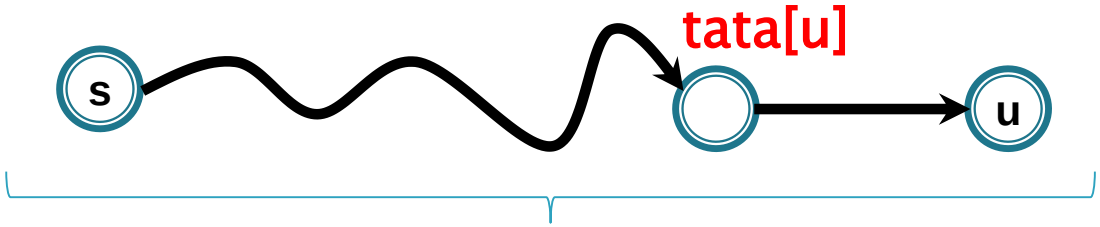
$$\delta(s,v) = \min\{\delta(s,u) + w(u,v) \mid uv \in E,\ \delta(s,u) < \delta(s,v)\ ?\ \}$$

Algoritmul lui DIJKSTRA – Vârfurile considerate în funcție de distanța **estimată** față de sursă, la fiecare pas este vizitat vârful cel mai apropiat de sursă și extinse drumurile din el

Caz neponderat – Vârfurile considerate în ordinea dată de BFS = ordine crescătoare în raport cu distanța față de sursă

Drumuri minime de sursă unică s

- **Idei comune**: **Pe parcursul algoritmului fiecare vârf are asociate informațiile:**
  - **d[u]** – etichetă de distanță
  - **tata[u]**



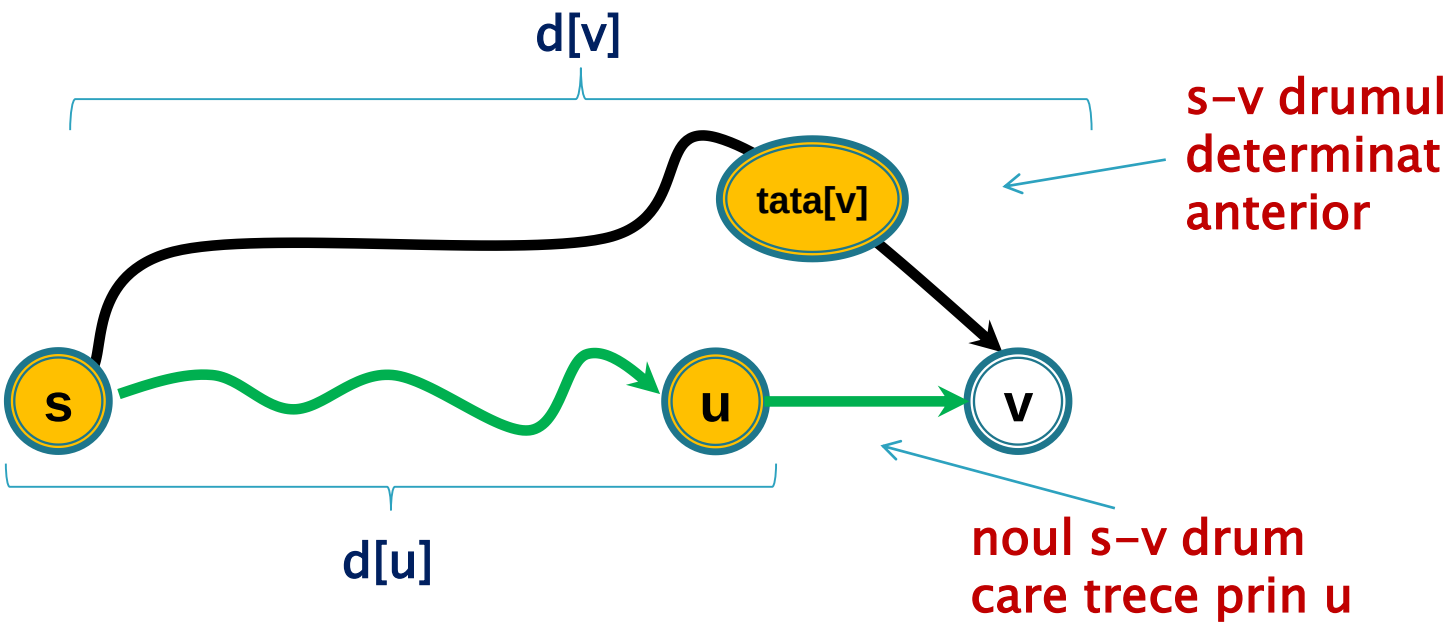
d[u] = costul minim al unui drum de la s la u descoperit până la acel moment

tata[u] = **predecesorul** lui u pe drumul de cost minim de la s la u descoperit până la acel moment

Drumuri minime de sursă unică s

- **Relaxarea unui arc (u, v)** = a verifica dacă d[v] poate fi îmbunătățit extinzând drumul minim deja găsit de la s la u cu arcul uv

```
dacă d[u] + w(u,v) < d[v] atunci
      d[v] = d[u] + w(u,v);
     tata[v] = u
```



Algoritmi – G=(V, E) graf orientat

G – neponderat
Parcurgere lățime BF

BF(s)

```
coada C ← ∅;
adauga(s, C)
```

G – ponderat, ponderi >0
Algoritmul lui Dijkstra

Dijkstra(s)

```
Q ← V
{se putea incepe doar cu Q ← {s}
+vector viz; v∈Q ⇔ v nevizitat}
```

G – ponderat fără circuite

DAGS(s)

```
SortTop ← sortare_topologica(G)
```

Algoritmi – G=(V, E) graf orientat

G – neponderat Parcurgere lățime BF	G – ponderat, ponderi >0 Algoritmul lui Dijkstra	G – ponderat fără circuite
BF(s)	**Dijkstra(s)**	**DAGS(s)**
coada C ← ∅; adauga(s, C)	Q ← V {se putea incepe doar cu Q ← {s} +vector viz; v∈Q ⇔ v nevizitat}	SortTop ← sortare_topologica(G)
pentru fiecare u∈V d[u]=∞;tata[u]=viz[u]=0 viz[s]← 1; d[s] ← 0	pentru fiecare u∈V d[u] = ∞; tata[u]=0 d[s] = 0	pentru fiecare u∈V d[u] = ∞; tata[u]=0 d[s] = 0

Algoritmi – G=(V, E) graf orientat

G – neponderat
Parcurgere lățime BF

```
BF(s)

coada C ← ∅;
adauga(s, C)

pentru fiecare u∈V
  d[u]=∞;tata[u]=viz[u]=0

viz[s]← 1; d[s] ← 0

cat timp C ≠ ∅
   u ←  extrage(C);

   pentru fiecare uv∈E
```

G – ponderat, ponderi >0
Algoritmul lui Dijkstra

```
Dijkstra(s)

Q ← V
{se putea incepe doar cu Q ← {s}
+vector viz; v∈Q ⇔ v nevizitat}

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

cat timp Q ≠ ∅
   u = extrage(Q) vârf cu eticheta
        d minimă

   pentru fiecare uv∈E
```

G – ponderat fără circuite

```
DAGS(s)

SortTop ← sortare_topologica(G)

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

pentru fiecare u ∈ SortTop


   pentru fiecare uv∈E
```

Algoritmi – G=(V, E) graf orientat

G – neponderat
Parcurgere lățime BF

BF(s)

```
coada C ← ∅;
adauga(s, C)

pentru fiecare u∈V
  d[u]=∞;tata[u]=viz[u]=0

viz[s]← 1; d[s] ← 0

cat timp C ≠ ∅
   u ←  extrage(C);

   pentru fiecare uv∈E
       daca viz[v]=0
          d[v] ← d[u]+1
          tata[v] ← u
          adauga(v, C)
          viz[v] ← 1

scrie d, tata
```

G – ponderat, ponderi >0
Algoritmul lui Dijkstra

Dijkstra(s)

```
Q ← V
{se putea incepe doar cu Q ← {s}
+vector viz; v∈Q ⇔ v nevizitat}

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

cat timp Q ≠ ∅
   u = extrage(Q) vârf cu eticheta
         d minimă

   pentru fiecare uv∈E
      daca ~~v∈Q~~ si d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u
            repara(v,Q)

scrie d, tata
```

G – ponderat fără circuite

DAGS(s)

```
SortTop ← sortare_topologica(G)

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

pentru fiecare u ∈ SortTop

   pentru fiecare uv∈E
       daca d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u

scrie d, tata
```

Algoritmi – G=(V, E) graf orientat

G – neponderat
Parcurgere lățime BF

BF(s)

```
coada C ← ∅;
adauga(s, C)

pentru fiecare u∈V
  d[u]=∞;tata[u]=viz[u]=0

viz[s]← 1; d[s] ← 0

cat timp C ≠ ∅
   u ←  extrage(C);

   pentru fiecare uv∈E
       daca viz[v]=0
          d[v] ← d[u]+1
          tata[v] ← u
          adauga(v, C)
          viz[v] ← 1

scrie d, tata
```

O(n+m)

G – ponderat, ponderi >0
Algoritmul lui Dijkstra

Dijkstra(s)

```
Q ← V
{se putea incepe doar cu Q ← {s}
+vector viz; v∈Q ⇔ v nevizitat}

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

cat timp Q ≠ ∅
   u = extrage(Q) vârf cu eticheta
        d minimă

   pentru fiecare uv∈E
      daca v∈Q si d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u
            repara(v,Q)

scrie d, tata
```

O(m log(n)) / O(n^2) – ca Prim

G – ponderat fără circuite

DAGS(s)

```
SortTop ← sortare_topologica(G)

pentru fiecare u∈V
    d[u] = ∞; tata[u]=0

d[s] = 0

pentru fiecare u ∈ SortTop

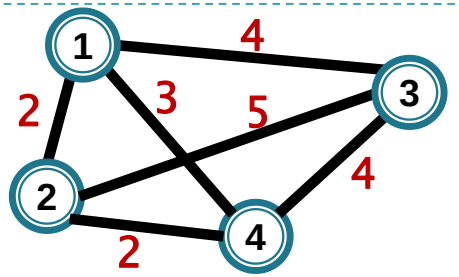
   pentru fiecare uv∈E
       daca d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u

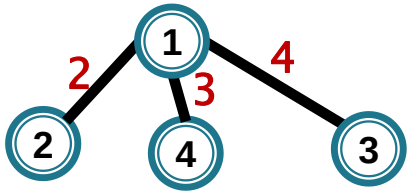
scrie d, tata
```

O(n+m)

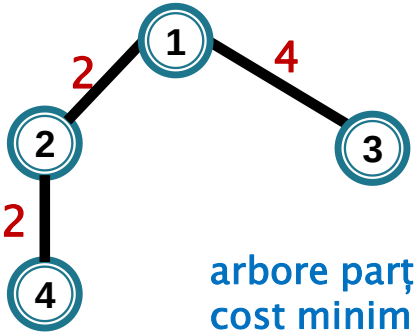
Drumuri minime din s ⇒ **arbore de drumuri minime (distanțe) din s**

≠ **arbore parțial de cost minim** – minimizează costul total





arbore al drumurilor minime față de 1



arbore parțial de cost minim

Drumuri minime din s ⇒ **arbore de drumuri minime (distanțe) din s**

≠ **arbore parțial de cost minim** – minimizează costul total

G-(ne)orientat ponderat,
ponderi >0

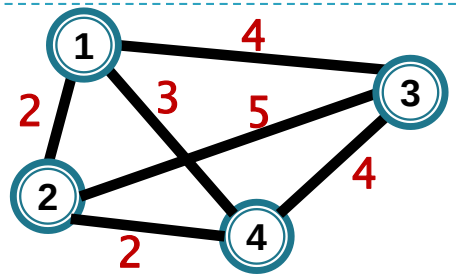
Drumuri minime din s

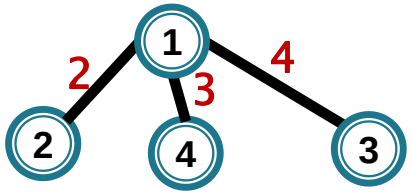
Algoritmul lui Dijkstra

G- neorientat ponderat
ponderi reale

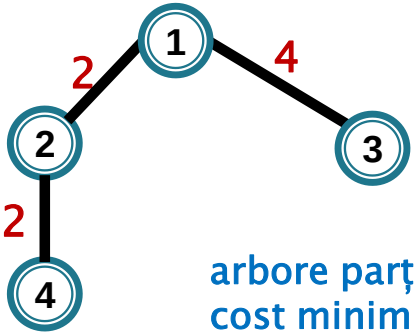
Arbore parțial de cost minim

Algoritmul lui Prim





arbore al drumurilor minime față de 1



arbore parțial de cost minim

Drumuri minime din s ⇒ arbore de drumuri minime (distanțe) din s

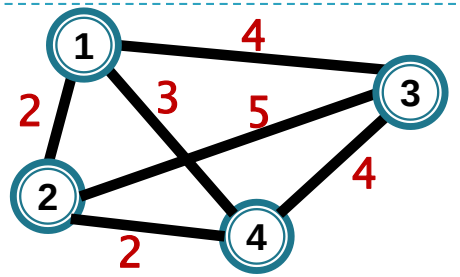
≠ **arbore parțial de cost minim** – minimizează costul total

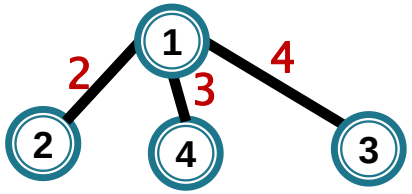
G-(ne)orientat ponderat, ponderi >0

Drumuri minime din s

Algoritmul lui Dijkstra

```
Dijkstra(s)
(min-heap) Q ← V
pentru fiecare u∈V
    d[u] = ∞; tata[u]=0
d[s] = 0
cat timp Q ≠ ∅
   u = extrage(Q) vârf cu eticheta
       d minimă
   pentru fiecare uv∈E
      daca ~~v∈Q~~ si d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u
            repara(v,Q)
scrie d, tata
```





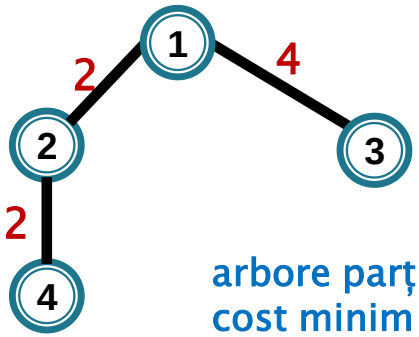
arbore al drumurilor minime față de 1

G- neorientat ponderat ponderi reale

Arbore parțial de cost minim

Algoritmul lui Prim

```
Prim(s)
(min-heap) Q ← V
pentru fiecare u∈V
    d[u] = ∞; tata[u]=0
d[s] = 0
cat timp Q ≠ ∅
   u = extrage(Q) vârf cu
       eticheta d minimă
   pentru fiecare uv∈E
      daca v∈Q si w(u,v)<d[v]
            d[v] = w(u,v)
            tata[v] = u
            repara(v,Q)
scrie (u, tata[u]), pentru u≠s
```



arbore parțial de cost minim

Drumuri minime din s ⇒ arbore de drumuri minime (distanțe) din s

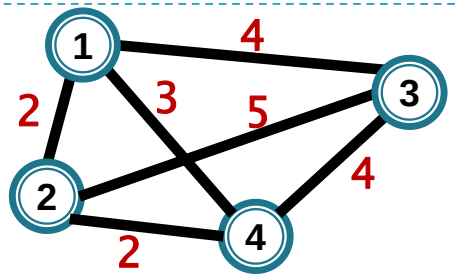
≠ **arbore parțial de cost minim** – minimizează costul total

G-(ne)orientat ponderat, ponderi >0

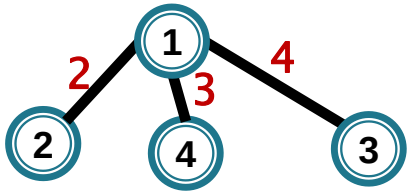
Drumuri minime din s

Algoritmul lui Dijkstra

```
Dijkstra(s)
(min-heap) Q ← V
pentru fiecare u∈V
    d[u] = ∞; tata[u]=0
d[s] = 0
cat timp Q ≠ ∅
   u = extrage(Q) vârf cu eticheta
       d minimă
   pentru fiecare uv∈E
      daca ~~v∈Q~~ si d[u]+w(u,v)<d[v]
            d[v] = d[u]+w(u,v)
            tata[v] = u
            repara(v,Q)
scrie d, tata
```



G



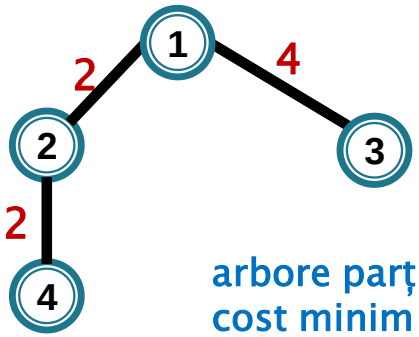
arbore al drumurilor minime față de 1

G- neorientat ponderat ponderi reale

Arbore parțial de cost minim

Algoritmul lui Prim

```
Prim(s)
(min-heap) Q ← V
pentru fiecare u∈V
    d[u] = ∞; tata[u]=0
d[s] = 0
cat timp Q ≠ ∅
   u = extrage(Q) vârf cu
       eticheta d minimă
   pentru fiecare uv∈E
      daca v∈Q si w(u,v)<d[v]
            d[v] = w(u,v)
            tata[v] = u
            repara(v,Q)
scrie (u, tata[u]), pentru u≠s
```



arbore parțial de cost minim